

Accelerating Fault Detection in Cyber-Physical Systems Using SmartNICs and DPDK

Samia Choueiri*, Sergio Elizalde*, Ali Mazloun*, Ali AlSabe[†], Jose Gomez[†], Elie Kfoury*, Jorge Crichigno*

*College of Engineering and Computing, University of South Carolina (USC)

[‡]College of Sciences and Engineering, University of South Carolina Aiken (USCA)

[†]Katz School of Business, Fort Lewis College, USA.

Email: {choueiri, elizalde, amazloun}@email.sc.edu, ali.alsabeh@usca.edu, jgomez@fortlewis.edu, ekfoury@email.sc.edu, jcrichigno@cec.sc.edu

Abstract—Cyber-physical systems (CPS) require continuous fault monitoring under strict latency constraints, where delayed detection can compromise control-loop stability and system reliability. This paper presents a SmartNIC-accelerated fault detection framework for Modbus/TCP CPS that performs flow-level feature extraction and Random Forest (RF) inference directly in the network data plane. The proposed system offloads a pre-trained RF model to an NVIDIA BlueField-3 SmartNIC and executes inference using the Data Plane Development Kit (DPDK), enabling low-latency, in-network classification of normal and faulty behavior. Through offline experimentation, we identify a lightweight feature set, window configuration, and model size that balance detection accuracy with inference efficiency. We further analyze feature importance and model explainability, and evaluate online accuracy, inference latency, and memory footprint for multiple model configurations. Experimental results show that the proposed approach achieves around 90% online detection accuracy with less than 100 μ s inference latency, demonstrating that RF inference can be efficiently executed in-network on SmartNICs for real-time CPS fault detection.

Index Terms—Cyber-physical systems, Fault Detection, SmartNICs, Random Forest, DPDK.

I. INTRODUCTION

Cyber-physical systems (CPS) underpin modern critical infrastructure such as energy, water, manufacturing, and transportation. They integrate physical processes with control logic executed on Programmable Logic Controllers (PLCs) and rely on industrial communication networks connecting field devices and Human–Machine Interfaces (HMIs). Traditionally, Operational Technology (OT) networks were isolated from Information Technology (IT) systems; however, increasing IT/OT convergence for remote monitoring, analytics, and centralized management has expanded connectivity and system complexity [1]. While this convergence improves operational efficiency, it also introduces new fault pathways. For example, IT-induced network latency, congestion, or misconfigurations can disrupt time-sensitive control traffic, leading to control-loop instability, actuator misbehavior, or sensor anomalies that resemble physical faults. Such faults can propagate rapidly across cyber and physical components, making timely and

reliable fault detection a critical safety challenge in modern CPS [1].

SmartNICs are next-generation network interface cards that enable customizable packet processing and offload networking and security functionalities from the host CPU to the data plane. Prior work has demonstrated their effectiveness in accelerating network monitoring and intrusion detection in resource-constrained and latency-sensitive environments, including IoT systems [2].

Motivated by the limitations of conventional CPS fault detection systems and the opportunities offered by SmartNIC-based in-network processing, this paper presents a SmartNIC-based fault detection framework for Modbus/TCP-based CPS, focusing on low-latency, in-network inference using a pre-trained Machine-Learning (ML) model. The proposed system offloads a Random Forest (RF) classifier to an NVIDIA BlueField-3 SmartNIC and executes inference directly within a DPDK-enabled datapath. Fault detection is performed using parsed industrial sensor readings from network packets under varying feature extraction approaches. By executing RF inference on the SmartNIC, the framework minimizes host CPU utilization, reduces data movement overhead, and enables accurate low-latency fault detection directly in the CPS data plane. The source code of this paper is publicly available [3].

The main contributions of this work are as follows:

- A novel framework that incorporates SmartNICs to accelerate the fault detection process in CPS. The framework operates at the data plane layer and is orders of magnitude faster than general-purpose servers.
- A SmartNIC in-network RF model that performs fault-tolerance detection at line rate. The model is trained on optimal features that reduce latency while maintaining high accuracy.
- A SHAP-based explainability analysis that provides insight into feature importance and supports interpretable fault detection in safety-critical CPS environments.
- A hardware model selection and online evaluation on an NVIDIA BlueField-3 SmartNIC, measuring classification accuracy, inference latency, and memory footprint under realistic traffic conditions.

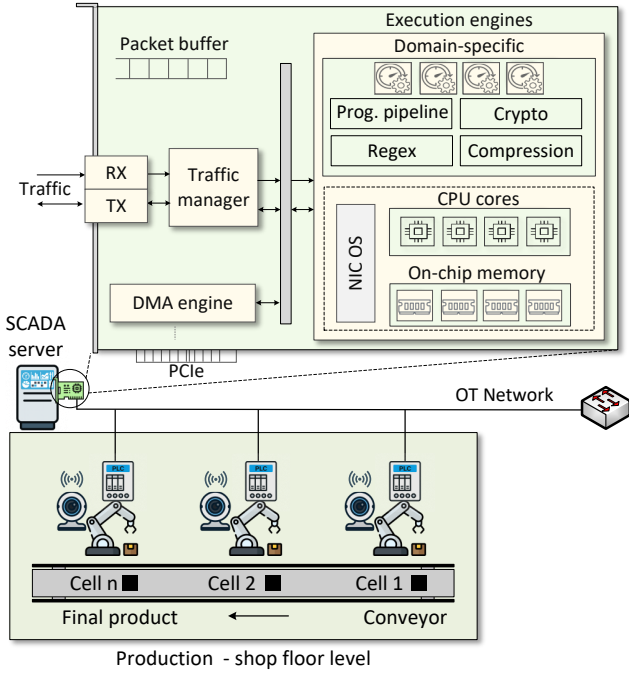


Fig. 1: CPS architecture showing PLC-controlled industrial cells connected via an OT network to a SCADA server equipped with a SmartNIC. The SmartNIC includes programmable data-plane components used for high-performance packet processing.

The remainder of this paper is structured as follows. Section II introduces the fundamental concepts of CPS and SmartNIC technology. Section III reviews related work on data plane and ML for cybersecurity in ICS. Section IV describes the architecture of the proposed hybrid detection framework. Finally, Section V reports the performance benchmarks, and Section VI summarizes the main findings and outlines future research.

II. BACKGROUND

A. Cyber-Physical Systems

CPSs with Supervisory Control and Data Acquisition (SCADA) represent the most widespread deployments in critical infrastructure sectors such as manufacturing, energy, and water treatment. These systems typically follow a hierarchical architecture that separates supervisory and physical process functions across different layers. At the supervisory layer, the SCADA server provides system monitoring, visualization, and high-level control. The control layer consists of Programmable Logic Controllers (PLCs), which execute real-time control logic and communicate with field devices (robots and machines) using industrial protocols such as Modbus/TCP. The adoption of Industry 4.0 concepts, including real-time data analytics and remote supervision, has increased reliance on IP-based networking within OT environments [4].

As a result, SCADA servers now serve as central aggregation points for high-rate sensor data and control traffic originating from multiple PLCs across the production floor, as shown in Fig. 1. Consequently, there is a growing need

for high-performance network components capable of handling protocol processing, traffic steering, and inline analysis without burdening the host CPU. Therefore, the SCADA server with a SmartNIC provides a promising platform to meet these requirements by enabling programmable, data-plane acceleration within OT networks.

B. Overview of SmartNICs and DPDK

SmartNICs are emerging devices that integrate programmable computing resources into the network interface. In general, the architecture of SmartNICs combines general-purpose CPUs with domain-specific processing engines [5]. This architecture may vary from one vendor's design to another. Fig. 1 presents the proposed SmartNIC abstraction used in our CPS architecture, reflecting capabilities commonly available in modern SmartNICs. Network traffic received at the RX NIC port is sent to the traffic manager or NIC switch. The SmartNIC includes execution engines composed of CPU cores for general-purpose tasks and DSAs for high-performance applications. On-chip memory is used to store flow state and intermediate data.

These components work together to offload processing from the host CPU and accelerate networking and security functions. Developers can program the SmartNIC using frameworks such as DPDK to gain direct access to packet queues, flow tables, and eSwitch functions [6]. This capability enables low-latency implementation of security, telemetry, and ML-based analytics directly within the network interface.

III. RELATED WORK

A. Programmable Data Planes in OT Systems

Recent research has increasingly leveraged programmable data planes to satisfy the stringent latency, determinism, and security requirements of CPS and OT networks. P4-based approaches focus on enforcing security and control logic directly within network switches, enabling line-rate deep packet inspection, stateful validation of legacy industrial protocols such as Modbus/TCP, and ultra-low-latency reaction to safety-critical events [7], [8]. While effective at high throughput, these solutions are constrained by hardware-specific pipelines and limited programmability, restricting their applicability across heterogeneous OT deployments. Complementary work explores eBPF-based packet processing to provide greater flexibility at the network edge, supporting dynamic function chaining, runtime reconfiguration, and complex stateful logic for resilience and asset discovery [9]. However, eBPF executions remain software-based and may struggle to deliver predictable performance under high traffic loads.

Our approach offloads packet processing tasks to a SmartNIC to leverage hardware programmability while employing DPDK as a kernel-bypass mechanism for fast user-space packet processing. This hybrid design aims to merge the deterministic performance of hardware acceleration with the adaptability of software-based processing.

B. ML for ICS Security and Resilience

Fault detection in cyber-physical systems is previously addressed using supervised Machine Learning (ML) models that classify multi-sensor data into normal and faulty states. Existing approaches range from distributed deep learning pipelines using auto-encoders and SVMs [9], to gradient-boosted decision trees for industrial robotics [10], and ML classifiers in CPS [11], [12]. While these methods achieve high detection accuracy, they typically rely on centralized execution on general-purpose CPUs or complex multi-stage inference pipelines, introducing non-negligible computational overhead and limiting their suitability for time-sensitive OT networks. Moreover, most prior work evaluates fault detection offline or at the application layer, without considering the cost of transporting high-rate industrial traffic and sensor data to the host for analysis. In contrast, our approach offloads RF inference directly to a SmartNIC using DPDK, enabling low-latency, deterministic fault detection within the network data plane and better aligning ML inference with the real-time requirements of CPS environments.

IV. PROPOSED FRAMEWORK

The proposed system follows a two-stage workflow that decouples offline model preparation from online in-network inference. As shown in Fig. 2, in the offline phase, raw CPS traces are processed on the host to extract flow-level statistical features of Modbus traffic. These features are used to train a RF classifier to distinguish normal operation from fault conditions. The trained model is then serialized into a compact

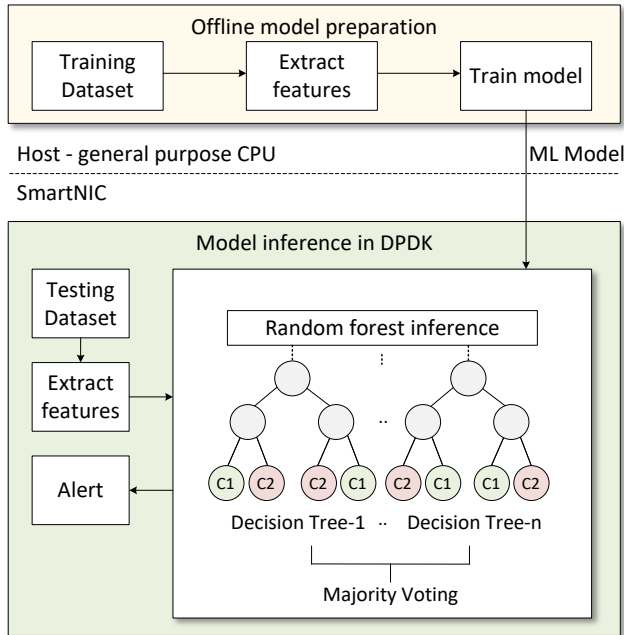


Fig. 2: Proposed system architecture. Features are extracted offline to train a RF model on the host, which is then offloaded to the SmartNIC for in-network inference.

TABLE I: Dataset used for normal operation and fault detection

Class	Network traffic (pkts)		PLCs
	Modbus/TCP	Syslog	Sensor readings
Normal	2863138	1128018	6534
Fault	1152195	535654	3806
Total	4015333	1663672	10340

JSON representation that encodes the structure and parameters of individual decision trees.

In the online phase, the model is offloaded to the SmartNIC and executed directly in the data plane using DPDK. RF inference is performed on the testing data by evaluating each decision tree independently, and the final classification is obtained through majority voting across all trees. Based on the inferred class, the system triggers an alert for detected faults, enabling low-latency and host-independent fault detection within the network.

A. Dataset

The proposed detection framework is evaluated using the publicly available dataset presented in [13]. The dataset was generated within an extended CPS environment and integrates three synchronized data sources: (i) physical process sensor readings from two PLCs, (ii) system logs collected from PLCs and HMIs, and (iii) Modbus/TCP network traffic captured in PCAP format. All data streams are timestamp-aligned, enabling correlation between process-level behavior and communication events.

Two categories of operational scenarios were considered: normal operation and system faults. Normal operation corresponds to baseline process execution. System fault scenarios consist of five injected fault types: sensor drift, tank leak, overheating caused by PLC delays, valve malfunction, and memory corruption. Table I summarizes the composition of the dataset used in this work, including Modbus/TCP traffic, Syslog messages, and PLC sensor readings for normal and fault classes.

B. Feature Engineering

Sensor measurements are parsed directly from Modbus packets and grouped into bidirectional communication flows. For each flow, only the first packets within a fixed window are considered to capture early behavioral characteristics to minimize feature extraction latency. From the extracted sensor values and packet timestamps, an initial set of eight statistical features is computed, including the minimum, maximum, mean, range, slope, and inter-arrival time statistics (min_iat , max_iat , and $mean_iat$). This baseline feature set is designed to capture both value distributions and short-term dynamics with minimal computational overhead.

To evaluate whether additional signal processing improves detection accuracy, a second feature set of eight features is derived by transforming the sensor signals using Exponentially

Algorithm 1: RF-Based Fault Classification

Input: Feature vector x (NUM_FEATURES), RF model
Output: Predicted fault class

```

1 Init: Load trained RF model; each tree represented as a set
  of nodes
2 foreach decision tree  $T$  in the forest do
3   node  $\leftarrow$  root of  $T$ 
4   while node is not a leaf do
5      $f \leftarrow$  node.feature
6      $\theta \leftarrow$  node.threshold
7     if  $x[f] \leq \theta$  then
8       node  $\leftarrow$  left_child(node)
9     else
10      node  $\leftarrow$  right_child(node)
11   vote[node.class_label]++
12 predicted_class  $\leftarrow$  argmax(vote)
13 return predicted_class

```

Weighted Moving Average (EWMA) and Cumulative Sum (CUSUM) techniques. EWMA provides a smoothed estimate of the signal trend and is computed as

$$z_t = \alpha x_t + (1 - \alpha)z_{t-1}, \quad (1)$$

where x_t is the sensor value at time t and $\alpha \in (0, 1]$ controls the smoothing factor. CUSUM captures persistent deviations from a reference mean μ_0 through cumulative positive and negative sums with a sensitivity parameter k :

$$C_t^+ = \max(0, C_{t-1}^+ + (x_t - \mu_0 - k)), \quad (2)$$

$$C_t^- = \min(0, C_{t-1}^- + (x_t - \mu_0 + k)), \quad (3)$$

C. In-network RF Implementation in DPDK

After offline training, the selected RF model is deployed on the SmartNIC and executed entirely within the DPDK data plane. To enable efficient in-network inference, the trained model is serialized into a JSON representation that encodes the structure of each decision tree and leaf class labels. During initialization, the DPDK application safely parses this representation, enforces memory constraints, and constructs compact, heap-allocated node arrays for each tree. This one-time setup allows subsequent packet processing to perform inference without dynamic memory allocation or host CPU involvement.

Each sample is evaluated independently by every decision tree in the forest. As shown in Algorithm 1, inference proceeds by traversing each tree from the root to a leaf node based on a sequence of feature-threshold comparisons. At each internal node, the feature value is compared against a stored threshold to determine whether the traversal follows the left or right child. Once a leaf node is reached, the corresponding class label is emitted as the tree's vote. The final prediction is obtained through majority voting across all trees, yielding a binary classification of normal or faulty behavior.

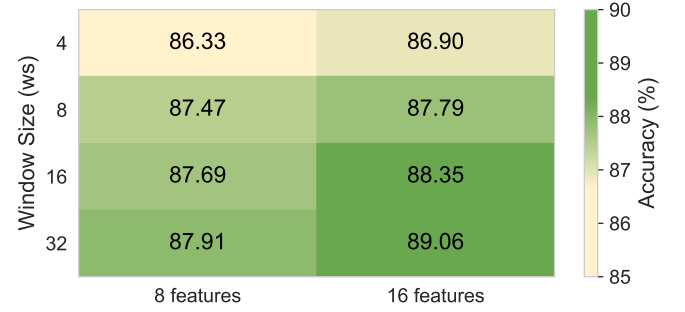


Fig. 3: Impact of feature dimensionality and window size on RF accuracy, comparing the baseline 8-feature set with the extended 16-feature set.

V. IMPLEMENTATION AND EVALUATION

This section evaluates the proposed SmartNIC-based fault detection system through a series of offline and online experiments. We analyze the impact of feature dimensionality and model configuration on detection accuracy, inference latency, and memory footprint, and assess the performance of the selected RF model when executed in the data plane on the SmartNIC.

A. Evaluating the Impact of Window Size and Number of Features

We first evaluate the impact of feature dimensionality and window size on classification accuracy by training RF models using either the baseline set of 8 statistical features or the extended set of 16 features that include EWMA- and CUSUM-based signals. For each configuration, features are extracted from only the first packets of each flow using window sizes of 4, 8, 16, and 32 packets. These window sizes are suitable for time-critical fault detection and demonstrate a high confidence level in the adopted RF model. As shown in Fig. 3, increasing the window size and feature count provides only marginal accuracy improvements. While the 16-feature configuration achieves slightly higher accuracy at larger window sizes, the overall gain is limited. In contrast, both the number of extracted features and the window size have a direct impact on feature extraction and inference latency. Given the absence of a significant accuracy advantage, we select the baseline 8-feature configuration with a window size of 8 packets as the most suitable trade-off between accuracy and low-latency in-network deployment.

B. Evaluating Feature Importance and Explainability

To better understand the decision-making behavior of the trained RF model, we analyze feature importance using SHAP (SHapley Additive exPlanations). This analysis focuses on the selected baseline set of eight features and provides insight into how individual features contribute to the classification of normal and faulty behavior.

Fig. 4 presents a SHAP summary plot, where each point represents the contribution of a feature value to the model output for a single sample. The horizontal axis shows the

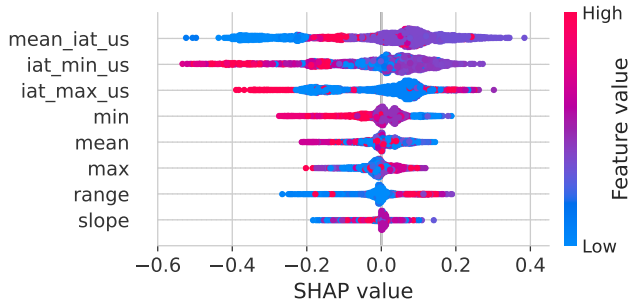


Fig. 4: SHAP summary plot showing the contribution of the selected eight features to the RF model's predictions.

SHAP value, indicating whether a feature pushes the prediction toward the fault class (positive values) or toward the normal class (negative values). The color of each point corresponds to the magnitude of the feature value, with warmer colors indicating higher values and cooler colors indicating lower values. Features are ordered from top to bottom by their overall impact on the model.

The figure shows that inter-arrival time features dominate the model's decisions, with the mean, minimum, and maximum inter-arrival times exhibiting the strongest influence on classification. Variations in these timing-related features consistently shift the model output, highlighting their effectiveness in capturing abnormal communication patterns. In contrast, value-based statistical features such as minimum, maximum, and mean sensor values contribute more modestly, while trend-oriented features such as slope and range have a comparatively smaller impact.

Overall, this analysis confirms that the selected feature set aligns well with the underlying behavior of Modbus traffic, where timing irregularities play a key role in distinguishing faults from normal operation. Moreover, the SHAP-based explainability provides confidence that the model relies on interpretable and physically meaningful features, which is essential for deployment in safety-critical CPS environments.

C. Evaluating the Impact of the Model Size

Before offloading the RF model to the SmartNIC, we evaluate the impact of key hyperparameters on detection performance in order to select suitable model configurations. Specifically, we vary the number of trees (i.e., estimators) in the forest from $\{1, 5, 10, 25, 50\}$ and the maximum depth per tree from $\{2, 4, 8, 16, 32\}$. Model performance is assessed using the Receiver Operating Characteristic Area Under the Curve (ROC-AUC) metric.

ROC-AUC measures the model's ability to distinguish between normal and faulty classes across all possible classification thresholds. A higher ROC-AUC indicates stronger separability, with values closer to 1.0 reflecting more reliable discrimination independent of a fixed decision threshold. This makes ROC-AUC particularly suitable for evaluating fault detection models under varying operating conditions.

In addition to ROC-AUC, we analyzed offline classification accuracy across the same hyperparameter space. The results

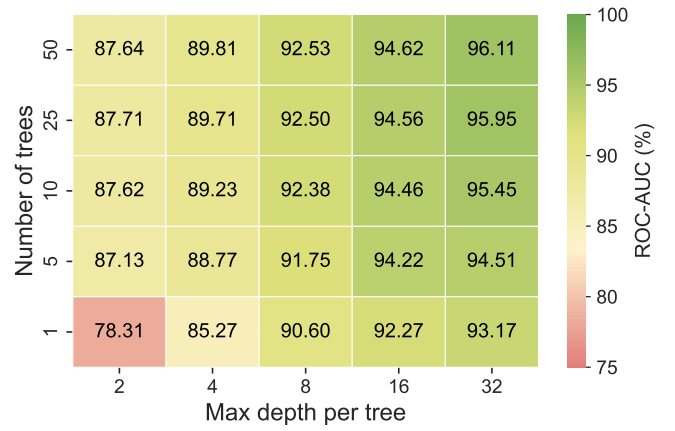


Fig. 5: ROC-AUC performance of RF models for varying numbers of trees and maximum tree depths.

show that accuracy generally increases with tree depth, as deeper trees capture more complex decision boundaries. However, accuracy improvements begin to plateau as the number of estimators grows, with diminishing gains observed beyond a moderate ensemble size.

Fig. 5 summarizes the ROC-AUC results across different model sizes. The results show that increasing the model size (number of trees and maximum depth per tree) yields only marginal improvements in classification performance beyond moderate depths. Based on these observations, we fix the maximum depth per tree to 32 and select multiple models with varying numbers of trees for deployment on the SmartNIC. These configurations are subsequently offloaded and evaluated online to analyze the trade-offs between accuracy, inference latency, and memory footprint in the data plane.

D. Evaluating the Online Accuracy, Inference Time, and Memory Footprint

To evaluate online performance, the selected RF models with a fixed maximum tree depth of 32 and varying numbers of trees ($\{1, 5, 10, 25, 50\}$) are serialized as JSON files and offloaded to the SmartNIC. Each model is executed entirely in the SmartNIC. For each configuration, we measure online classification accuracy, inference latency, and memory footprint.

Fig. 6 shows the cumulative distribution functions (CDFs) of inference latency for the five model sizes. Larger models generally incur higher inference latency, as classification requires traversing a greater number of decision trees. This trend is reflected in both the increasing mean latency and the larger standard deviation observed as the number of trees grows. However, the CDFs also reveal that a subset of packets processed by larger models can still experience relatively low inference times.

This variability arises from the structure of RF. Although the maximum depth is fixed, individual trees may terminate early depending on the feature values encountered during traversal. Samples that reach leaf nodes closer to the root require fewer comparisons, resulting in shorter inference paths even in large

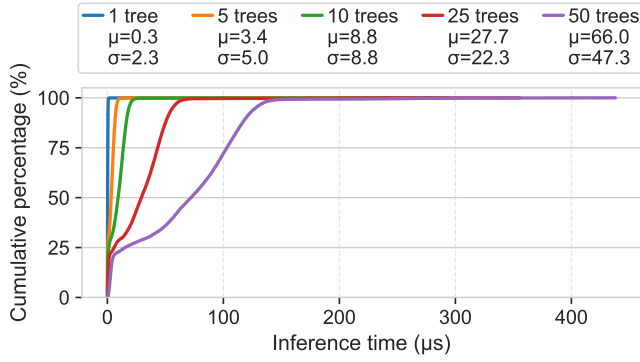


Fig. 6: CDF of inference latency for RF models with different numbers of trees, executed in the SmartNIC data plane. Mean (μ) and standard deviation (σ) are reported for each configuration.

ensembles. While samples that traverse deeper branches across many trees incur higher latency. As the number of trees increases, this effect becomes more pronounced, leading to wider latency distributions and higher variance.

Overall, these results highlight the trade-off between model size and inference determinism in the data plane. While increasing the number of trees improves classification robustness, it also increases both the average inference time and its variability. This motivates a careful selection of model size when deploying RF inference with hardware constraints.

In addition to inference latency, we analyze the impact of model size on online classification accuracy and memory footprint. As the number of trees increases, online accuracy consistently improves due to a stronger ensemble effect from majority voting. Larger ensembles are more robust to noisy or ambiguous samples, yielding higher detection accuracy during live traffic processing, but at the cost of increased memory usage, as each additional tree must be stored and traversed in SmartNIC memory.

Table II summarizes online performance and compares it with offline training accuracy. While offline accuracy improves only marginally with larger models, online accuracy shows more pronounced gains, particularly when moving from small to medium ensemble sizes. This gap underscores the importance of evaluation under realistic in-network conditions, where timing variations and partial flow visibility affect classification outcomes.

VI. CONCLUSION AND FUTURE WORK

This paper presented a SmartNIC-accelerated fault detection framework for CPS that offloads RF inference directly into the network data plane using DPDK. Through a systematic evaluation of feature dimensionality, window size, and model configuration, we identified a lightweight feature set and model architecture that balances detection accuracy with low-latency execution. Experimental results on an NVIDIA BlueField-3 SmartNIC demonstrate that in-network inference can achieve high accuracy while significantly reducing inference latency and host CPU involvement, making the proposed approach suitable for real-time CPS environments. As future work, we

TABLE II: Online and offline performance comparison for RF models.

Trees	Inference Time (μ s)		Accuracy (%)		Memory Footprint (MiB)
	Mean	Std. Dev.	Offline	Online	
1	0.3	2.3	85.86	78.13	0.78
5	3.4	5.0	88.17	83.76	3.50
10	8.8	8.8	88.75	85.20	7.19
25	27.7	22.3	89.22	87.43	18.08
50	66.0	47.3	89.25	88.36	36.15

plan to explore adaptive and incremental learning to handle evolving system behavior without full retraining, as well as hardware-assisted acceleration of tree traversal and feature extraction, and support for additional industrial protocols.

ACKNOWLEDGMENT

The work was supported by the National Science Foundation, under awards 2403360 and 2400729. The authors would also like to acknowledge the FABRIC [14] team.

REFERENCES

- [1] K. Stouffer, K. Stouffer, M. Pease, C. Tang, T. Zimmerman, V. Pillitteri, S. Lightman, A. Hahn, S. Saravia, A. Sherule *et al.*, “Guide to operational technology (OT) security,” 2023.
- [2] S. Elizalde, A. AlSabe, A. Mazloun, S. Choueiri, E. Kfoury, J. Gomez, and J. Crichigno, “A survey on security applications with smartnics: Taxonomy, implementations, challenges, and future trends,” *Journal of Network and Computer Applications*, p. 104257, 2025.
- [3] samiachoueiri Github, “DPDK-ICS-RF,” [Online]. Available: <https://github.com/samiachoueiri/DPDK-ICS-RF>, 2026.
- [4] H. Lasi, P. Fettke, H.-G. Kemper, T. Feld, and M. Hoffmann, “Industry 4.0,” *Business & information systems engineering*, 2014.
- [5] E. F. Kfoury, S. Choueiri, A. Mazloun, A. AlSabe, J. Gomez, and J. Crichigno, “A comprehensive survey on SmartNICs: architectures, development models, applications, and research directions,” *IEEE Access*, 2024.
- [6] S. Choueiri, A. Mazloun, S. Elizalde, E. F. Kfoury, and J. Crichigno, “Volumetric DDoS Attack Detection and Mitigation using P4-DPDK,” in *2025 IEEE International Black Sea Conference on Communications and Networking (BlackSeaCom)*, 2025, pp. 1–6.
- [7] R. Samson Raj and D. Jin, “Leveraging data plane programmability towards a policy-driven in-network security framework for industrial control systems,” in *Proceedings of the 16th ACM International Conference on Future and Sustainable Energy Systems*, 2025, pp. 152–163.
- [8] F. E. R. Cesen and C. E. Rothenberg, “Offloading Robotic and UAV applications to the network using programmable data planes,” in *2023 IEEE Conference on Network Function Virtualization and Software Defined Networks (NFV-SDN)*. IEEE, 2023, pp. 207–212.
- [9] S. U. Jan, Y. D. Lee, and I. S. Koo, “A distributed sensor-fault detection and diagnosis framework using machine learning,” *Information Sciences*, 2021.
- [10] P. Doshi, N. Daftary, H. Jani, and A. Nair, “CatBoost-driven anomaly detection in industrial robotic arms Using CASPER dataset,” *IEEE Sensors Letters*, 2026.
- [11] S. Elizalde, S. Choueiri, A. Mazloun, J. Gomez, E. Kfoury, and J. Crichigno, “Accelerating anomaly detection in industrial control systems using SmartNICs and DPDK,” in *IEEE Consumer Communications & Networking Conference (CCNC)*, Las Vegas, NV, USA, Jan. 2026.
- [12] A. F. Amiri, H. Oudira, A. Chouder, and S. Kichou, “Faults detection and diagnosis of PV systems based on machine learning approach using random forest classifier,” *Energy Conversion and Management*, 2024.
- [13] D. Sudyana, “ICS Dataset,” Sep. 2025. [Online]. Available: <https://doi.org/10.5281/zenodo.17165850>
- [14] I. Baldin, A. Nikolich, J. Griffioen, I. I. S. Monga, K.-C. Wang, T. Lehman, and P. Ruth, “Fabric: A national-scale programmable experimental network infrastructure,” *IEEE Internet Computing*, vol. 23, no. 6, pp. 38–47, 2020.